

Where Does the Loss Come From?

Disentangling Token Pruning and Quantization in Compressed Vision–Language Document Retrieval

T Rithik Krishna*, Amgovath Navanitha*, Badavath Akhila*, E. Sathiya Lakshmi†

Department of Emerging Technologies, Mahatma Gandhi Institute of Technology (Autonomous)

Affiliated to Jawaharlal Nehru Technological University Hyderabad, Gandipet, Hyderabad 500075, Telangana, India

*{23261a6753, 23261a6704, 23261a6709}@mgit.ac.in †sathiyalakshmi_et@mgit.ac.in

Abstract—Vision–language retrievers such as ColPali remove optical character recognition from the document search pipeline by embedding rendered page images directly, but they replace one dense vector per page with roughly a thousand patch vectors, inflating the index by two to three orders of magnitude. The standard remedy is to prune visual tokens and to quantize the survivors, and systems that do both report large end-to-end compression figures. Those figures do not say which of the two stages the retrieval quality was actually paid to. We build OptiVision RAG, a model-agnostic compression layer that applies pixel-space saliency pruning, embedding-space redundancy collapsing and one-bit quantization to any ColBERT-style visual retriever, and we run a controlled ablation in which the corpus is encoded once and every configuration replays over identical cached vectors. On a 60-page corpus with 72 queries under exact brute-force MaxSim scoring, the two stages are found to be sharply asymmetric. Both pruning stages together give a $3.5\times$ token reduction for 3.9% of nDCG@5 and leave the baseline ranking largely intact (Kendall $\tau = 0.87$); one-bit quantization alone costs 12.1% of nDCG@5 without discarding a single token and drives τ down to 0.585. Every configuration that includes binary codes lands in a narrow 85–88% retention band regardless of how aggressively tokens are pruned, so the token budget is not the operative variable once the codec is fixed. Replacing the one-bit codec with int8 yields $14.2\times$ compression at 97.1% retention and adds no measurable ranking distortion on top of pruning. We conclude that for multi-vector visual retrieval the quantizer, not the token count, is the binding constraint, and that reported compression ratios should be accompanied by a per-stage attribution.

Index Terms—document retrieval, vision–language models, late interaction, token pruning, binary quantization, index compression, ColPali

I. Introduction

Retrieval over scanned and born-digital documents has traditionally been staged: optical character recognition (OCR) extracts text, a layout parser recovers structure, a text encoder produces embeddings, and a vector database serves them. Each stage is a source of error, and the first is the worst — OCR degrades on scans, handwriting, stamps and dense tables, and the layout, figures and formatting that a human reader uses to locate information are discarded before the retriever ever sees them.

Vision–language retrievers avoid the pipeline entirely. ColPali [1] renders each page as an image, splits it

into a grid of patches, and embeds every patch with a vision–language backbone, scoring a query against a page with the late-interaction MaxSim operator inherited from ColBERT [2]. No text is ever extracted, and the approach is state of the art on the ViDoRe benchmark.

The cost is the index. A page is no longer one vector but roughly a thousand patch vectors of 128 dimensions each. At float32 this is approximately 448 KB per page, so a one-million-page archive requires an index on the order of 448 GB, which must remain resident for search to be fast. The accuracy is affordable; the memory is not.

Two families of remedy exist and are usually applied together. Token reduction removes patch vectors before they are stored, on the observation that a document page is mostly blank paper; it descends from work on accelerating vision transformers, such as DynamicViT [6] and token merging [7]. Quantization shrinks each surviving vector, from product quantization [4] to the one-bit hashing used in dense passage retrieval [5]. Combining them multiplies the savings, and a system that prunes to a quarter of the tokens and stores the rest as one-bit codes can honestly advertise a compression ratio above $100\times$.

That headline number is also where the reporting usually stops. It says how small the index became and how much retrieval quality was lost in total, but not which stage the quality was spent on. The distinction is not academic: the two stages have entirely different cost structures and entirely different upgrade paths. If most of the loss belongs to pruning, the productive research direction is a better saliency model. If most of it belongs to the codec, pruning harder is nearly free and the effort belongs in the quantizer instead. A single end-to-end figure cannot distinguish these cases, and the two prescriptions are opposites.

This paper performs that attribution. Our contributions are:

- 1) A model-agnostic compression layer. OptiVision RAG applies pixel-space saliency pruning, embedding-space redundancy collapsing and configurable quantization to the patch embeddings of any ColBERT-style visual retriever, leaving the encoder unmodified. Pruning decisions are made from trans-

parent pixel statistics rather than a learned model, so the behaviour is interpretable and the operating point is a dial rather than a retrained network.

- 2) A controlled per-stage ablation. The corpus is encoded once and every configuration replays over identical cached vectors under exact brute-force MaxSim, so a change in the metric can be caused only by the compression setting. Alongside absolute retrieval metrics we report Kendall τ against the uncompressed baseline’s own ranking, which isolates whether compression changed the retriever’s mind independently of whether the retriever was right.
- 3) The finding that the two stages are sharply asymmetric. Pruning is close to free and pruning harder is nearly free on top of that; one-bit quantization costs 12.1% of nDCG@5 while discarding no tokens at all, and every binary configuration lands in the same narrow retention band. We identify two defensible operating points that follow from this and argue that compression ratios for multi-vector retrieval should be published with a per-stage attribution.

II. Related Work

A. Late-interaction and visual retrieval

ColBERT [2] introduced late interaction: rather than compressing a document into one vector, it retains one vector per token and scores a query by summing, over query tokens, the maximum similarity against any document token. ColBERTv2 [3] reduced the resulting storage with residual compression over centroids. ColPali [1] carried the mechanism into the visual domain, treating image patches as the document tokens, and Document Screenshot Embedding [9] showed that page screenshots are a viable universal document representation. These systems establish the accuracy that motivates our work and the index size that motivates compressing it.

B. Token reduction

DynamicViT [6] learns to discard uninformative tokens during inference and token merging [7] combines similar ones, both aimed at throughput for classification backbones. The objective differs from ours in a way that matters: a classifier needs only its pooled representation to survive, whereas a late-interaction retriever needs the surviving vectors to preserve a ranking over pages. A token that never wins a MaxSim maximum is free to remove; a token that decides a close ordering is not, and classification accuracy cannot detect the difference.

C. Quantization for retrieval

Product quantization [4] and binary hashing [5] are standard for compressing dense embeddings, and modern vector databases expose both alongside approximate nearest-neighbour structures such as HNSW [8]; Qdrant [10] additionally supports multi-vector MaxSim comparators with binary quantization and rescoreing.

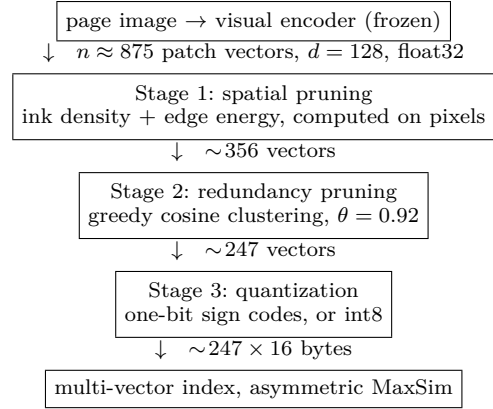


Fig. 1. The compression layer. Vector counts are the measured means over the evaluation corpus. Only the three middle stages are ours; the encoder and the index are used as published.

These techniques are typically evaluated on single-vector retrieval, where each document contributes one vector and quantization error affects one score. Under MaxSim the error enters through a maximum over hundreds of vectors per page, which is a different statistical regime, and it is applied uniformly with no notion of which tokens matter.

D. Positioning

The individual components we use are not novel, and we do not claim them. What is missing from the literature is a controlled attribution of quality loss between pruning and quantization when both are applied to the same multi-vector visual index. Systems report the product of the two savings; we report the ratio of the two costs.

III. Method

A. Overview

OptiVision RAG sits between an unmodified visual encoder and the vector index (Fig. 1). A page is rendered to an image and encoded into $n \approx 875$ patch vectors of dimension $d = 128$. A saliency map computed directly from the page pixels selects which grid cells survive; a redundancy pass in embedding space collapses near-duplicate survivors; a quantizer compresses what remains; and the result is stored as a multi-vector page entry scored by MaxSim. The encoder is never fine-tuned and never sees the compression, which is what makes the layer transferable across ColPali, ColQwen2 and ColSmol checkpoints.

B. Stage 1: pixel-space saliency pruning

The premise that a page is mostly paper can be tested on the pixels, before the model runs and at negligible cost. For each cell of the patch grid we compute two signals on the greyscale page. Ink density is the mean positive deviation below the paper level, where the paper level is estimated as the 95th percentile of intensity rather than a fixed white point, so that yellowed or photocopied scans are not read as ink everywhere. Edge energy is the mean



Fig. 2. Spatial pruning on a page from the evaluation corpus. Left: the rendered page. Centre: the pixel-space saliency map. Right: the retained patch mask, green for kept and red for discarded — 233 of 768 grid patches (30%) survive, and after redundancy collapsing 233 of 875 vectors (27%) are stored. The retained region tracks the text block, and the margins, which occupy most of the page, cost nothing.

absolute first difference in both axes, which responds to glyph strokes, rules, table lines and stamp boundaries. Both are normalised by a high percentile rather than the maximum, so a single dark speck — dust, a punch hole, a staple shadow — cannot squash the rest of the page toward zero. The saliency of cell (r, c) is

$$s_{rc} = \frac{w_{\text{ink}} \hat{I}_{rc} + w_{\text{edge}} \hat{E}_{rc}}{w_{\text{ink}} + w_{\text{edge}}}, \quad (1)$$

with $w_{\text{ink}} = 0.6$ and $w_{\text{edge}} = 0.4$. Ink alone drops faint handwritten annotation; edges alone retain scanner noise on an empty margin; the weighted sum is robust to both and costs microseconds. Cells with $s_{rc} > \tau_{\text{blank}} = 0.02$ are kept, and the resulting mask is dilated by one cell in 4-connectivity, because glyphs rarely stop at a patch boundary and the ascender of a heading routinely falls into a neighbouring cell whose own density is below threshold. A floor of 8 cells guarantees that a nearly empty page remains retrievable. An alternative mode keeps the top k cells by saliency, which fixes the per-page budget — this is the mode used for the sweep in Section V-B, since a fixed budget is what index-size guarantees require. Fig. 2 shows the two modes’ effect on a representative page.

C. Stage 2: embedding-space redundancy pruning

Stage 1 removes patches with no content. Stage 2 removes patches with duplicate content: the interior of a filled table cell, the middle of a thick rule, a run of uniform texture inside a photograph. These are salient to a pixel detector and yet contribute nothing to a MaxSim score. We visit the survivors in descending saliency order and greedily form clusters of vectors within cosine $\theta = 0.92$ of the representative, replacing each cluster with its renormalised mean.

This is close to free in ranking terms for a structural reason. MaxSim takes, per query vector, the maximum similarity over document vectors. If two document vectors lie within cosine θ of one another, the maximum over the

pair and the similarity to their renormalised mean differ by an amount bounded by their angular spread, which is small by construction — so collapsing a tight cluster barely moves the score while removing one stored vector per duplicate. The argument does not extend to loose clusters, which is why θ is set high.

D. Stage 3: quantization and asymmetric scoring

A 128-dimensional float32 vector occupies 512 bytes; retaining only the sign of each dimension gives 128 bits, or 16 bytes — a flat $32\times$ on everything the pruner allowed through. These vectors are L2-normalised with approximately zero-centred dimensions, so $\text{sign}(\mathbf{d})$ preserves the orthant of $\mathbf{d}$ and discards only its position within that orthant.

We score asymmetrically: the document side is binarised, the query side is not. A fully symmetric scheme reduces MaxSim to a popcount Hamming distance and is cheaper still, but it discards the query magnitudes — and those are precisely the part that can be afforded, since a query contributes on the order of twenty vectors against a corpus of millions. Asymmetric scoring costs one unpack and one float matrix product and recovers most of the ranking quality; the symmetric variant is retained only for comparison. For a query matrix Q and a page’s code matrix C ,

$$\text{MaxSim}(Q, C) = \sum_i \max_j \mathbf{q}_i \cdot \text{sign}(\mathbf{d}_j). \quad (2)$$

We also evaluate an int8 codec at $4\times$. Because the components of an L2-normalised 128-dimensional vector concentrate near $1/\sqrt{d}$ — the observed mean absolute component is 0.071 and the observed maximum 0.363 — the textbook $\text{round}(127v)$ mapping, which assumes the data spans $[-1, 1]$, leaves roughly two thirds of the available levels permanently unused. Rescaling by the range the data actually occupies costs no additional storage and reduces the mean cosine error on real vectors from 3.28×10^{-4} to 8.20×10^{-5} .

E. Memory accounting

One implementation detail is load-bearing for the claims in this paper. A naive scorer decodes the packed index into a float32 matrix before computing similarities, which restores the full $d \times 4$ bytes per vector at query time and returns the entire saving — an index of 2.82 KB per page on disk demanded 90.12 KB per page in RAM to search. The storage claim was true while the system-level claim behind it was not. We therefore score in blocks sized against a memory budget, releasing each block before the next is allocated; peak resident memory then tracks the budget (measured at $0.93\times$ of it) and is independent of corpus size, while producing scores bit-identical to the cached path. All index sizes reported below are computed from the pipeline’s own byte accounting rather than from files on disk, so they are neither inflated by container overhead nor deflated by filesystem compression.

IV. Experimental Setup

A. Configuration

The encoder is vidore/colSmol-256M (ColIdefics3), float32, on CPU; hardware is a laptop with 8 GB of DDR4 and no GPU. The corpus is 60 generated pages (30 documents of 2 pages) rendered at A4, 150 dpi. Vector dimension is 128 and the encoder emits 875 vectors per page before compression (768 page-grid patches, 64 thumbnail, 43 instruction). Every run uses seed 1337.

B. Queries and ground truth

There are 72 queries in two deliberately different families. Sixty precise queries pair a subject with a unique identifier and have exactly one relevant page; these check that compression has not broken retrieval outright. Twelve topical queries name only a subject shared by several pages, so the metric depends on ordering a group of near-identical candidates — which is where compression damage becomes visible. Ground truth is exact by construction.

C. Controls

Two choices make the ablation a controlled experiment rather than a leaderboard.

Exact scoring. Every quality figure comes from exhaustive brute-force MaxSim, not from an approximate index. An ANN structure would contribute its own recall loss and confound the attribution we are trying to make. The Qdrant backend is tested separately and is the deployment path, not the measurement path.

One encoding pass. The corpus is encoded once into a cache and every ablation row replays over those identical vectors. Nothing but the compression setting differs between rows, so a difference in a metric can have no other cause. Re-running the benchmark reproduces the reported figures exactly; the pipeline is deterministic given a fixed corpus and encoder.

D. Metrics

We report nDCG@5 with binary relevance (the standard ViDoRe metric), Recall@1 and Hit@5, index bytes per page, and the compression ratio against the uncompressed float32 index. We additionally report retention — nDCG@5 as a fraction of the baseline row — and Kendall τ -b against the baseline’s own ranking. The last is the load-bearing metric here. Absolute quality answers “is this retriever any good?”, which depends mostly on the choice of encoder; τ answers “did compression change the retriever’s mind?”, which is the only question this layer controls.

V. Results

Table I is the full ablation and Fig. 3 plots it. Three findings follow, and the third is the one that matters.

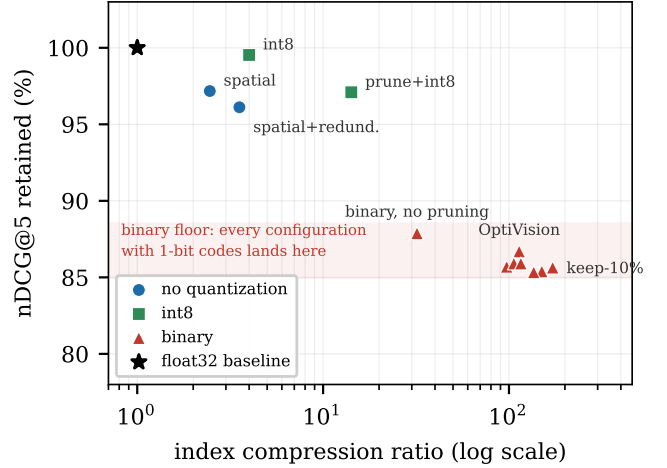


Fig. 3. Compression ratio against nDCG@5 retention, by quantizer. The points separate by codec, not by token count: everything with one-bit codes falls into the shaded band between 85% and 88% retention whether it prunes nothing or prunes to a tenth, while the unquantized and int8 configurations stay above 96%.

A. Finding 1: pruning is close to free

Removing blank patches costs 2.8% of nDCG@5 for a $2.5\times$ token reduction, and adding redundancy collapsing brings this to $3.5\times$ for 3.9%. Rank agreement with the uncompressed baseline remains high throughout ($\tau = 0.87\text{--}0.94$), meaning the pruned index does not merely score similarly but largely preserves the ordering. Notably, spatial-only matches the baseline exactly on Recall@1 (0.5694): discarding 59% of the vectors does not cost a single top-1 hit. This is the central premise of the approach and it holds.

B. Finding 2: pruning harder is nearly free too

The token-budget sweep (Fig. 4) is almost flat. Keeping the top 10% of patches — 163 vectors per page — retains 85.6% of nDCG@5, while keeping the top 50% — 288 vectors, a $1.8\times$ larger index — retains 85.7%. The 0.1 point separating them is well inside the noise of 72 queries. Once quantization is in play the token budget barely moves quality, so the aggressive setting is not a trade-off at all: it is close to a free $1.8\times$.

C. Finding 3: the loss lives in the quantizer

binary-only loses 12.1% of nDCG@5 without discarding a single token, and its τ of 0.585 says it substantially re-orders results rather than merely rescaling scores. Against this, int8-only costs 0.5% for $4\times$ with $\tau = 0.973$. Every configuration that includes one-bit codes lands in a narrow 85–88% retention band regardless of how much pruning is applied — the binary floor dominates everything else in the table.

The cleanest statement of the asymmetry is a comparison of τ across two rows. spatial+redundancy and prune+int8 have identical τ of 0.866: adding int8 on top

TABLE I

Ablation over the compression pipeline. 60 pages, 72 queries, ColSmol-256M, exact MaxSim. Rows share one encoding pass, so every difference is attributable to the compression setting alone. Retain is nDCG@5 relative to the baseline row; τ is Kendall τ -b against the baseline’s own ranking.

Variant	What it isolates	Tok/pg	KB/pg	Compr.	nDCG@5	R@1	Hit@5	Retain	τ
baseline-float32	reference: every patch, full precision	875.0	448.00	1.0×	0.7823	0.5694	0.9722	100.0%	1.000
Pruning alone (no quantization)									
spatial-only	blank-patch pruning	356.1	182.32	2.5×	0.7602	0.5694	0.9444	97.2%	0.935
spatial+redundancy	both pruning stages	246.8	126.34	3.5×	0.7519	0.5139	0.9722	96.1%	0.866
Quantization alone (no pruning)									
int8-only	scalar quantization	875.0	112.00	4.0×	0.7787	0.5556	0.9722	99.5%	0.973
binary-only	one-bit quantization	875.0	14.00	32.0×	0.6875	0.4167	0.9444	87.9%	0.585
Both stages									
prune+int8	quality-first operating point	246.8	31.59	14.2×	0.7596	0.5278	0.9722	97.1%	0.866
optimvision	size-first operating point	246.8	3.95	113.5×	0.6782	0.4028	0.9583	86.7%	0.606
optimvision-aggressive	fixed 25% token budget	186.3	2.98	150.3×	0.6680	0.3611	0.9722	85.4%	0.602
Token-budget sweep, all with one-bit codes									
keep-50%	top 50% by saliency	288.1	4.61	97.2×	0.6704	0.4167	0.9444	85.7%	0.548
keep-40%	top 40% by saliency	263.6	4.22	106.2×	0.6721	0.4167	0.9583	85.9%	0.561
keep-30%	top 30% by saliency	241.6	3.87	115.9×	0.6721	0.4028	0.9583	85.9%	0.588
keep-20%	top 20% by saliency	206.1	3.30	135.9×	0.6676	0.3750	0.9722	85.3%	0.586
keep-10%	top 10% by saliency	162.9	2.61	171.8×	0.6700	0.4167	0.9306	85.6%	0.551

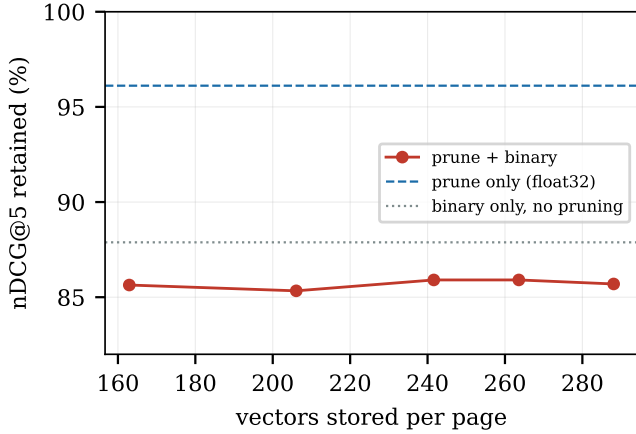


Fig. 4. The token-budget sweep, all rows using one-bit codes. Varying the retained fraction from 10% to 50% moves nDCG@5 retention by 0.6 points, while the gap to the unquantized pruned index (dashed) is over 10 points. The budget is not the operative variable; the codec is.

of both pruning stages introduces no measurable ranking distortion whatsoever, while cutting the index by a further 4×. All of the reordering in the full pipeline is attributable to the one-bit codec.

Read together, the three findings invert the intuition the system was designed around. The pruning stages, which are the elaborate part of the pipeline and consumed most of the engineering, are the safe part; the quantizer, which is a single sign test, is where the quality goes.

D. Two operating points

Table II summarises the practical consequence. For an archive where the index must fit in a fixed small

TABLE II

The two defensible configurations that follow from the ablation.

Goal	Configuration	Compr.	Retain	τ
smallest index	prune + binary	113.5×	86.7%	0.606
quality per byte	prune + int8	14.2×	97.1%	0.866

memory and the retriever feeds a downstream reader that tolerates imperfect ordering, prune + binary at 113.5× is appropriate. Where ranking fidelity matters — a shortlist shown directly to a user — prune + int8 delivers 14.2× at 97.1% retention and undistorted ordering, which is the better default. The 8× difference in index size between them buys 10.4 points of retention, and which side of that exchange is correct is a deployment question rather than a research one.

One row deserves an explicit caveat. prune+int8 scores 0.7596 against spatial+redundancy’s 0.7519, so quantization appears to improve nDCG@5 slightly. At 72 queries this is noise, and it should be read as “int8 is free here”, not as “int8 helps”.

E. Latency

Query latency falls with the token count, from 3.25 ms at the float32 baseline to 0.70 ms at keep-10% on the 60-page corpus, since MaxSim cost is linear in stored vectors. We do not emphasise these figures: at this corpus size latency is dominated by fixed overhead, and the exact index used for measurement is not the structure a deployment would use. Query encoding, at 65.7 ms, dominates search entirely at this scale.

VI. Discussion

A. Why the binary floor exists

For random unit vectors the expected cosine between $\mathbf{d}$ and $\text{sign}(\mathbf{d})/\sqrt{d}$ is $\sqrt{2/\pi} \approx 0.798$, and the distortion is nearly the same for every document vector. Uniform distortion shifts scores far more than it reorders them, which is the usual argument for why binary quantization is safe. Our τ of 0.585 shows the argument is insufficient under MaxSim, and the reason is the maximum. A page score is a sum of per-query-vector maxima over hundreds of candidates; the identity of the $\arg\max$ is decided by small margins between the top few document vectors, and one-bit rounding perturbs exactly those margins. Distortion that is uniform in expectation is not uniform in its effect on an $\arg\max$, and this is why single-vector intuitions about binary quantization transfer poorly to late-interaction retrieval.

B. Implications

Three follow. First, reported compression ratios for multi-vector retrieval should be accompanied by a per-stage attribution, since the same end-to-end number can be reached by configurations with entirely different upgrade paths. Second, given a fixed one-bit codec, the token budget is close to a free parameter, so a system that has already committed to binary codes should prune as aggressively as its saliency detector allows. Third, the highest-value research direction for this class of system is not a better saliency model but a better codec: a two-bit or product-quantized document side sitting between binary ($32\times$) and int8 ($4\times$) would dominate both of our operating points if it recovers most of the 12% at around $16\times$. Rescoring a binary-retrieved shortlist with full-precision vectors for the top k pages only is the cheapest test of whether the loss is a ranking problem or a recall problem; our Hit@5 remains at 0.9583 for the full pipeline while nDCG@5 drops sharply, which suggests the relevant page is usually still in the shortlist and merely misordered — exactly the regime in which shortlist rescoring should recover most of the gap.

C. Limitations

We state these plainly, as they bound the claims above.

Corpus scale and provenance. Sixty generated pages with exact ground truth give a clean attribution but coarse metric resolution, and real scans add noise, skew and bleed-through that a generator does not. The storage ratios are scale-independent; the quality figures would tighten with more pages. Our harness includes a ViDoRe loader that treats real benchmark data identically, and running it is the immediate next step.

Encoder scale. ColSmol-256M is a 256M-parameter model, and its embeddings are plausibly less robust to one-bit rounding than ColPali-3B’s. Absolute quality here is below published ColPali figures. The compression behaviour is what we expect to transfer, since no stage

touches the model — but the magnitude of the binary penalty specifically should be treated as an upper bound until the same table is produced on ColPali with a GPU.

Scoring path. All quality numbers come from exact brute-force MaxSim, deliberately. A deployment on an approximate index would see additional recall loss from the ANN structure, which our figures do not include and were designed not to include.

Statistical power. With 72 queries, differences below roughly one point of nDCG@5 are not resolvable, which is why we decline to interpret the prune+int8 row’s apparent improvement and why the sweep is described as flat rather than as having a shape.

VII. Conclusion

We built a model-agnostic compression layer for vision-language document retrieval and used it to ask which of its two stages the retrieval quality is actually paid to. Under a controlled ablation in which the corpus is encoded once and only the compression setting varies, the two stages proved sharply asymmetric: both pruning stages together cost 3.9% of nDCG@5 for a $3.5\times$ token reduction and largely preserve the baseline ordering, while one-bit quantization costs 12.1% while discarding no tokens at all. Every binary configuration lands in the same 85–88% band regardless of token budget, and int8 on top of pruning leaves Kendall τ exactly unchanged at 0.866 while adding $4\times$ compression. The token count is therefore not the binding constraint in this pipeline; the codec is.

The practical output is two defensible operating points — $113.5\times$ at 86.7% retention, or $14.2\times$ at 97.1% with undistorted ranking — and the methodological one is that a compression ratio for multi-vector retrieval should not be published without saying which stage paid for it. Future work follows directly from the attribution: reproduce the table on ColPali-v1.3 over the ViDoRe benchmark, place a two-bit or product-quantized codec between the two current operating points, and test shortlist rescoring, which our Hit@5 figures suggest should recover much of the binary gap at negligible cost.

Acknowledgment

The authors thank the Department of Emerging Technologies, Mahatma Gandhi Institute of Technology, for supporting this work.

References

- [1] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot and P. Colombo, “ColPali: Efficient Document Retrieval with Vision Language Models,” in Proc. Int. Conf. on Learning Representations (ICLR), 2025.
- [2] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in Proc. 43rd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, 2020, pp. 39–48.
- [3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts and M. Zaharia, “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction,” in Proc. NAACL-HLT, 2022, pp. 3715–3734.

- [4] H. Jégou, M. Douze and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
- [5] I. Yamada, A. Asai and H. Hajishirzi, “Efficient Passage Retrieval with Hashing for Open-domain Question Answering,” in *Proc. ACL-IJCNLP*, 2021, pp. 979–986.
- [6] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou and C.-J. Hsieh, “DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 13937–13949.
- [7] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer and J. Hoffman, “Token Merging: Your ViT But Faster,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [8] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [9] X. Ma, S.-C. Lin, M. Li, W. Chen and J. Lin, “Unifying Multimodal Retrieval via Document Screenshot Embedding,” in *Proc. EMNLP*, 2024, pp. 6600–6611.
- [10] Qdrant, “Multivectors, Binary Quantization and Rescoring,” Qdrant Documentation. [Online]. Available: <https://qdrant.tech/documentation/>